

4-Week Roadmap for Drone Kernel Development

This roadmap is structured for a 4-person kernel team working on an STM32-first drone kernel intended to evolve into a reusable embedded kernel for future microcontrollers. The focus areas align with real-time drone system requirements, including fixed-priority scheduling, modular kernel APIs, safety mechanisms, and integration with sensor, control, and FSM teams.

Week 1 — Core Concepts and Baseline Alignment

Week 1 is assigned to Vikram, Aanand, and Ahmed. The goal is to bring them up to the current code level while exposing them to core kernel and STM32 concepts that directly support later scheduler, inter-task communication, and safety work.

Vikram, Aanand, and Ahmed

- Create a single-producer single-consumer lock-free queue using atomic operations as a first step toward kernel-side inter-task communication.
- Create a priority scheduler prototype that runs simple tasks in priority order without preemption, matching the current system baseline.
- Use a mutex to manage a shared variable in a small learning example so critical sections and blocking behavior are clearly understood before the team standardizes on atomics for selected paths.
- Send a simple "Hello" message on STM32 to validate board bring-up, flashing, and the basic firmware path.

Week 1 outcomes

- Shared understanding of task structure, scheduler basics, atomic operations, critical sections, and STM32 bring-up.
- Small reviewable artifacts that could be reused or refined in later kernel development.

Week 2 — Kernel Interfaces and Execution Model

The focus for Week 2 is to turn the current codebase into a cleaner kernel substrate with explicit system-call-style APIs and a defined execution model. Since this kernel will serve as the integration layer for sensor, FSM, and control teams, the API contract should be stable, minimal, and documented early.

Core work

- Define kernel-facing APIs for task creation, deletion policy if needed, task state transitions, delay/yield behavior, priority assignment, and TCB field updates. (almost done)
- Add context switching implementation details for ARM Cortex-M, including stack frame layout, save/restore flow, and the role of exception-based switching such as PendSV and related handlers.
- Establish the time-keeping system using SysTick or an equivalent periodic timer, with a clear kernel tick definition, task timing model, and basic delay/time accounting support.
- Write the first version of the kernel interface documentation so other teams know what services exist, what assumptions they must follow, and which features are not yet stable.

Recommended additions

- Define task state diagrams and TCB contents before expanding API surface, so implementation does not drift from the model.
- Add a basic trace or debug logging hook to capture task creation, dispatch, yield, and fault events for later validation.

Week 3 — Preemptive Scheduling and Inter-Task Communication

Week 3 should convert the current priority scheduler into a preemptive fixed-priority scheduler using Rate Monotonic Scheduling as the main policy for periodic real-time work, while retaining a secondary cooperative tier for non-critical or deferred work. This is a good fit for drone workloads because control and sensing tasks are naturally periodic and benefit from deterministic fixed-priority dispatch.

Core work

- Upgrade the scheduler to preemptive fixed-priority dispatch, with RMS-based priority assignment for periodic tasks and a secondary cooperative tier for lower-criticality work.
- Implement context-switch triggers from timer interrupts and voluntary yield points, and verify correctness under nested timing scenarios.
- Build the ring buffer as the first explicit inter-task communication primitive, keeping the design SPSC and lock-free where possible using atomic operations.
- Add scheduler validation artifacts such as timing checks, response-time observations, and a simple utilization review for planned periodic workloads.

Recommended additions

- Define which kernel events are safe in interrupt context versus task context, especially for queue push/pop and wake-up behavior.
- Reserve time to review priority inversion risk even if the immediate design prefers atomics, because future expansion may introduce shared structures that are not naturally lock-free.

Week 4 — Safety, Validation, and Integration Readiness

Week 4 should harden the kernel so it is reliable enough to support adjacent teams. Research on real-time drone systems consistently emphasizes deterministic execution, fault detection, and recovery support, which makes this week essential rather than optional.

Core work

- Override key ARM Cortex hardware handlers needed for kernel-level fault awareness, including fault reporting paths that help diagnose crashes and unexpected states.
- Configure and validate the STM32 Independent Watchdog so the system can recover from hangs or missed forward progress conditions.
- Implement stack watermarking to estimate worst-case stack usage and catch dangerous margin erosion before runtime failures appear in integrated testing.
- Build testing and validation infrastructure, including unit tests for scheduler and ring buffer behavior, fault-injection tests where practical, and integration test scenarios aligned to expected interactions with sensor, control, and FSM teams.
- Expand documentation into a usable kernel package: API reference, timing model notes, safety behavior, integration assumptions, and known limitations for the current release.

Recommended additions

- Define a minimum bring-up checklist for integration with other teams, including task registration rules, timing expectations, and fault-report capture.
- Add a short release-readiness review at the end of the week so the team can decide what is production-worthy and what remains experimental.

Wins and Risks

Using static allocation for shared state, TCBs, and the ring buffer reduces heap-related unpredictability and simplifies timing behavior in a real-time embedded kernel. Preferring atomic operations for SPSC data paths is also reasonable for low-overhead, non-blocking communication,

although mutex-based exercises still have value in Week 1 because they teach concurrency hazards and critical section thinking.

The main missing risk to watch is not memory allocation but schedule correctness under real drone timing. RMS is only useful when task periods, execution times, and preemption behavior are measured and reviewed, so validation should be treated as a first-class roadmap item rather than a final cleanup step.